# deep_asd_prediction

March 14, 2025

## 1 Deep ASD Prediction

The goal of this model is to develop a deep neural network capable of predicting if a subject was classified with Autism Spectrum Disorder (ASD) traits.

[ ]:
```
import pandas as pd

DATA_PATH = 'data/Autism_Prediction'
```

### 1.1 Part 1: Data analysis

- Load the data from kaggle
- Observe the types, distribution, and other qualities of the data

[1673]:
```
asd_toddlers_df = pd.read_csv(f'{DATA_PATH}/Toddler Autism dataset July 2018.
↪csv')
asd_toddlers_df = asd_toddlers_df.dropna()
asd_toddlers_df.rename(columns={'Class/ASD Traits ':'ASD_Traits'}, inplace=True)
```

#### 1.1.1 What kind of columns do we have available?

| Column Name | Description | Type |
|---|---|---|
| Case_no | An index | int |
| A1-A10 | Answers for questions A1-10 | int (binary 1 or 0) |
| Age_Mons | age of the subject in months | int |
| Qchat-10-Score (should be discarded ) | Tallied Qchat-10 score based on answers | int (1 - 10) |
| Sex | sex of the subject | string (‘f’ or ‘m’) |
| Ethnicity | ethnicity of subject | string |
| Jaundice | yellowing of the skin, eyes, and mucous membranes sometimes associated with autism | string (‘yes’ or ‘no’) |
| Who completed the test | This is always ‘family member’ since test is for toddlers | string |
| Family_mem_with_ASD | indicates if subject has family member with autism | string (‘yes’ or ‘no’) |

| Column Name | Description | Type |
|---|---|---|
| Class/ASD Traits | ASD traits or no ASD traits | string ('YES' or 'NO') |

I have chosen plotly to show some visualizations of the data. Each visualization is interactive, so feel free to hover and click to explore each diagram.

[1674]:
```
import plotly.express as px
from plotly.subplots import make_subplots
import plotly.graph_objects as go
```

[1675]:
```
# Sunburst chart
fig_sunburst = px.sunburst(
    asd_toddlers_df,
    path=['Ethnicity', 'Sex', 'ASD_Traits'],  # Hierarchy: Ethnicity -> Sex ->␣
 ↪ASD classification
    title="Sunburst of Ethnicity, Sex, and ASD Classification",
)
fig_sunburst.update_layout(
    title={'x': 0.5, 'xanchor': 'center', 'font': {'size': 20}},
    width=700,
    height=500,
)
fig_sunburst.show()
```

First is a sunburst diagram to see how our data is composed in terms of ethnicity, gender, and ASD classification (the target variable). We see that the majority of rows are from white males followed by asian males with latino, pacifica, native indian with the smallest representation. We must keep this bias of the data in mind before drawing any serious conclusions about a model's predictions.

---

[1676]:
```
fig_parallel = px.parallel_categories(
    asd_toddlers_df,
    dimensions=['Ethnicity', 'Jaundice', 'Family_mem_with_ASD', 'ASD_Traits'],
    title="Parallel Categories: Ethnicity, Jaundice, Family History, and ASD␣
 ↪Classification"
)
fig_parallel.show()
```

In this parallel categories plot we see how the different features ultimately flow into the ASD Traits target variable. Some of the biggest groups flowing into toddlers with ASD traits are asians with no jaundice and no family members with ASD and also white toddlers with no jaundice and no family members with ASD.

---

[1677]:
```
fig_violin_age = px.violin(
    asd_toddlers_df,
    x='ASD_Traits',
    y='Age_Mons',
    color='ASD_Traits',  # Same column used for coloring
    box=True,            # Show inner box plot
    points='all',        # Show all individual data points
    title='Age Distribution by ASD Classification (Violin Plot)'
)
fig_violin_age.show()
```

[1678]:
```
fig_treemap_ethnicity = px.treemap(
    asd_toddlers_df,
    path=['Ethnicity', 'ASD_Traits'],
    title='Treemap of Ethnicity and ASD Classification'
)
fig_treemap_ethnicity.show()
```

[1679]:
```
fig_bar_gender = px.histogram(
    asd_toddlers_df,
    x="Sex",
    color="ASD_Traits",
    barmode="group",
    color_discrete_map={"1": "#FF8042", "0": "#0088FE"},
    title="ASD Traits Distribution by Gender",
    labels={"Sex": "Gender", "ASD_Traits": "ASD Traits"}
)
fig_bar_gender.update_layout(
    xaxis_title="Gender",
    yaxis_title="Count",
    legend_title="ASD Traits",
    font=dict(size=12)
)
```

[1680]:
```
ethnicity_counts = asd_toddlers_df.groupby(['Ethnicity', 'ASD_Traits']).size().
 ↪reset_index(name='Count')

# Filter for major ethnicities (at least 30 records) for better visualization
major_ethnicities = asd_toddlers_df['Ethnicity'].
 ↪value_counts()[asd_toddlers_df['Ethnicity'].value_counts() >= 30].index.
 ↪tolist()
ethnicity_counts_filtered = ethnicity_counts[ethnicity_counts['Ethnicity'].
 ↪isin(major_ethnicities)]

# Calculate percentage of ASD Traits=Yes for each ethnicity
ethnicity_percentages = asd_toddlers_df[asd_toddlers_df['Ethnicity'].
 ↪isin(major_ethnicities)].groupby('Ethnicity').apply(
```

```
    lambda x: (x['ASD_Traits'] == 'Yes').mean() * 100
).reset_index(name='Percentage_Yes')

# Sort ethnicities by percentage of ASD Traits=Yes
ethnicity_percentages = ethnicity_percentages.sort_values('Percentage_Yes',␣
 ↪ascending=False)
sorted_ethnicities = ethnicity_percentages['Ethnicity'].tolist()

# Create the bar chart with sorted ethnicities
fig_bar_ethnicity = px.bar(
    ethnicity_counts_filtered,
    x="Ethnicity",
    y="Count",
    color="ASD_Traits",
    barmode="group",
    category_orders={"Ethnicity": sorted_ethnicities},
    color_discrete_map={"Yes": "#FF8042", "No": "#0088FE"},
    title="ASD Traits Distribution by Ethnicity",
    labels={"Ethnicity": "Ethnicity", "Count": "Count", "ASD_Traits": "ASD␣
 ↪Traits"}
)
fig_bar_ethnicity.update_layout(
    xaxis_title="Ethnicity",
    yaxis_title="Count",
    legend_title="ASD Traits",
    font=dict(size=12)
)
```

```
/var/folders/50/ps11njl93r52n622qd1nd_h80000gn/T/ipykernel_90580/4272123507.py:8
: DeprecationWarning:

DataFrameGroupBy.apply operated on the grouping columns. This behavior is
deprecated, and in a future version of pandas the grouping columns will be
excluded from the operation. Either pass `include_groups=False` to exclude the
groupings or explicitly select the grouping columns after groupby to silence
this warning.
```

[1681]:
```
fig_histo_q10_score = px.histogram(
    asd_toddlers_df,
    x="Qchat-10-Score",
    color="ASD_Traits",
    barmode="stack",
    nbins=11,
    color_discrete_map={"Yes": "#FF8042", "No": "#0088FE"},
    title="Q-Chat-10 Score Distribution by ASD Traits",
    labels={"Qchat-10-Score": "Q-Chat-10 Score", "ASD_Traits": "ASD Traits"}
```

```
)
fig_histo_q10_score.update_layout(
    xaxis_title="Q-Chat-10 Score",
    yaxis_title="Count",
    legend_title="ASD Traits",
    font=dict(size=12)
)
```

[1682]:
```
fig_bar_age = px.histogram(
    asd_toddlers_df,
    x="Age_Mons",
    color="ASD_Traits",
    barmode="group",
    color_discrete_map={"Yes": "#FF8042", "No": "#0088FE"},
    title="ASD Traits Distribution by Age (in Months)",
    labels={"Age_Mons": "Age (Months)", "ASD_Traits": "ASD Traits"}
)
fig_bar_age.update_layout(
    xaxis_title="Age (Months)",
    yaxis_title="Count",
    legend_title="ASD Traits",
    font=dict(size=12)
)
```

[1683]:
```
# Jaundice History
fig_jaundice = go.Figure()

jaundice_counts = asd_toddlers_df.groupby(['Jaundice', 'ASD_Traits']).size().
 ↪reset_index(name='Count')
jaundice_yes = jaundice_counts[jaundice_counts['ASD_Traits'] == 'Yes']
jaundice_no = jaundice_counts[jaundice_counts['ASD_Traits'] == 'No']

fig_jaundice.add_trace(
    go.Bar(x=jaundice_yes['Jaundice'], y=jaundice_yes['Count'], name='ASD␣
 ↪Traits: Yes', marker_color='#FF8042')
)
fig_jaundice.add_trace(
    go.Bar(x=jaundice_no['Jaundice'], y=jaundice_no['Count'], name='ASD Traits:␣
 ↪No', marker_color='#0088FE')
)

fig_jaundice.update_layout(
    title_text="Impact of Jaundice on ASD Traits",
    barmode='group',
    font=dict(size=12)
)
```

[1684]:
```
# Family History
fig_family = go.Figure()

family_counts = asd_toddlers_df.groupby(['Family_mem_with_ASD', 'ASD_Traits']).
 ↪size().reset_index(name='Count')
family_yes = family_counts[family_counts['ASD_Traits'] == 'Yes']
family_no = family_counts[family_counts['ASD_Traits'] == 'No']

fig_family.add_trace(
    go.Bar(x=family_yes['Family_mem_with_ASD'], y=family_yes['Count'],␣
 ↪name='ASD Traits: Yes', marker_color='#FF8042', showlegend=False)
)
fig_family.add_trace(
    go.Bar(x=family_no['Family_mem_with_ASD'], y=family_no['Count'], name='ASD␣
 ↪Traits: No', marker_color='#0088FE', showlegend=False)
)

fig_family.update_layout(
    title_text="Impact of Family History on ASD Traits",
    barmode='group',
    font=dict(size=12)
)
```

[1685]:
```
pivot_sex_age = pd.pivot_table(
    asd_toddlers_df,
    values='Case_No',
    index=['Sex'],
    columns=['Age_Mons', 'ASD_Traits'],
    aggfunc='count',
    fill_value=0
).stack(level=0).reset_index()

fig_scatter_traits = px.scatter(
    asd_toddlers_df,
    x="Age_Mons",
    y="Qchat-10-Score",
    color="ASD_Traits",
    symbol="Sex",
    title="Q-Chat-10 Score by Age, Sex, and ASD Traits",
    color_discrete_map={"Yes": "#FF8042", "No": "#0088FE"},
    labels={"Age_Mons": "Age (Months)", "Qchat-10-Score": "Q-Chat-10 Score",␣
 ↪"ASD_Traits": "ASD Traits", "Sex": "Gender"}
)
fig_scatter_traits.update_layout(
    xaxis_title="Age (Months)",
    yaxis_title="Q-Chat-10 Score",
    legend_title="ASD Traits",
```

```
    font=dict(size=12)
)
```

```
/var/folders/50/ps11njl93r52n622qd1nd_h80000gn/T/ipykernel_90580/3658983364.py:8
: FutureWarning:

The previous implementation of stack is deprecated and will be removed in a
future version of pandas. See the What's New notes for pandas 2.1.0 for details.
Specify future_stack=True to adopt the new implementation and silence this
warning.
```

[1686]:
```
correlation_data = []

for i in range(1, 11):
    question = f'A{i}'
    # Correlation between positive answer (1) and ASD_Traits=Yes
    positive_yes = asd_toddlers_df[(asd_toddlers_df[question] == 1) &␣
 ↪(asd_toddlers_df['ASD_Traits'] == 'Yes')].shape[0]
    positive_total = asd_toddlers_df[asd_toddlers_df[question] == 1].shape[0]

    # Calculate percentage of ASD_Traits=Yes among those who answered positively
    positive_yes_percentage = (positive_yes / positive_total * 100) if␣
 ↪positive_total > 0 else 0

    correlation_data.append({
        'Question': question,
        'Correlation_with_ASD': positive_yes_percentage
    })

correlation_df = pd.DataFrame(correlation_data)
correlation_df = correlation_df.sort_values('Correlation_with_ASD',␣
 ↪ascending=False)

fig_correlation = px.bar(
    correlation_df,
    x='Question',
    y='Correlation_with_ASD',
    color='Correlation_with_ASD',
    color_continuous_scale='RdBu_r',
    title="Correlation of Positive Responses with ASD Traits",
    labels={"Question": "Question", "Correlation_with_ASD": "% of ASD␣
 ↪Traits=Yes Among Positive Responses"}
)
fig_correlation.update_layout(
    xaxis_title="Question",
    yaxis_title="% of ASD Traits=Yes Among Positive Responses",
```

```
    font=dict(size=12)
)
```

[1687]:
```
age_qchat = asd_toddlers_df.groupby('Age_Mons')['Qchat-10-Score'].mean().
 ↪reset_index()
age_count = asd_toddlers_df.groupby('Age_Mons').size().reset_index(name='Count')
age_qchat = pd.merge(age_qchat, age_count, on='Age_Mons')

fig_scatter_q10_age = px.scatter(
    age_qchat,
    x="Age_Mons",
    y="Qchat-10-Score",
    size="Count",
    title="Average Q-Chat-10 Score by Age",
    labels={"Age_Mons": "Age (Months)", "Qchat-10-Score": "Average Q-Chat-10␣
 ↪Score", "Count": "Number of Children"}
)
fig_scatter_q10_age.update_layout(
    xaxis_title="Age (Months)",
    yaxis_title="Average Q-Chat-10 Score",
    font=dict(size=12)
)
```

[1688]:
```
questions_data = asd_toddlers_df[['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7',␣
 ↪'A8', 'A9', 'A10']].mean().reset_index()
questions_data.columns = ['Question', 'Positive_Response_Rate']
questions_data['Positive_Response_Rate'] =␣
 ↪questions_data['Positive_Response_Rate'] * 100  # Convert to percentage

fig_bar_questions = px.bar(
    questions_data,
    x='Question',
    y='Positive_Response_Rate',
    color='Positive_Response_Rate',
    color_continuous_scale='Viridis',
    title="Positive Response Rate for Individual Questions (A1-A10)",
    labels={"Question": "Question", "Positive_Response_Rate": "Positive␣
 ↪Response Rate (%)"}
)
fig_bar_questions.update_layout(
    xaxis_title="Question",
    yaxis_title="Positive Response Rate (%)",
    font=dict(size=12)
)
```

Below cell is for creating subplots of the above diagrams, only uncomment if you want to see groupings of plots.

[1689]:
```
# Grouping plots
# Group 1 - Ethnicity
# fig = make_subplots(
#     rows=2, cols=2,
#     subplot_titles=(
#         'Treemap of Ethnicity and ASD Classification',
#         'ASD Traits Distribution by Ethnicity',
#         'Sunburst of Ethnicity, Sex, and ASD Classification',
#         'Parallel Categories: Ethnicity, Jaundice, Family History, and ASD Classification'
#     ),
#     specs=[[{'type': 'bar'}, {'type': 'treemap'}], [{ 'type': 'sunburst' }, {'type': 'domain' }]]
# )
# for trace in fig_bar_ethnicity.data:
#     fig.add_trace(trace, row=1, col=1)

# for trace in fig_treemap_ethnicity.data:
#     fig.add_trace(trace, row=1, col=2)

# for trace in fig_sunburst.data:
#     fig.add_trace(trace, row=2, col=1)

# for trace in fig_parallel.data:
#     fig.add_trace(trace, row=2, col=2)

# fig.update_layout(title={'text': '', 'x': 0.5}, height=1000, width=2000)
# fig.show()

# Group 2 - Age
# fig = make_subplots(
#     rows=2, cols=2,
#     subplot_titles=(
#         'ASD Traits Distribution by Age (in Months)',
#         'Age Distribution by ASD Classification (Violin Plot)',
#         'Average Q-Chat-10 Score by Age',
#         'Q-Chat-10 Score by Age, Sex, and ASD Traits'
#     ),
#     specs=[[{'type': 'bar'}, {'type': 'violin'}], [{ 'type': 'scatter' }, {'type': 'scatter' }]]
# )
# for trace in fig_bar_age.data:
#     fig.add_trace(trace, row=1, col=1)

# for trace in fig_violin_age.data:
#     fig.add_trace(trace, row=1, col=2)
```

```
# for trace in fig_scatter_q10_age.data:
#     fig.add_trace(trace, row=2, col=1)

# for trace in fig_scatter_traits.data:
#     fig.add_trace(trace, row=2, col=2)

# fig.update_layout(title={'text': '', 'x': 0.5}, height=1000, width=2000)
# fig.show()

# Group 3 - Other features
# fig = make_subplots(
#     rows=2, cols=2,
#     subplot_titles=(
#         'ASD Traits Distribution by Gender',
#         'Q-Chat-10 Score Distribution by ASD Traits',
#         'Impact of Jaundice on ASD Traits',
#         'Impact of Family History on ASD Traits'
#     ),
#     specs=[[{'type': 'bar'}, {'type': 'violin'}], [{ 'type': 'scatter' }, {␣
 ↪'type': 'scatter' }]]
# )
# for trace in fig_bar_gender.data:
#     fig.add_trace(trace, row=1, col=1)

# for trace in fig_histo_q10_score.data:
#     fig.add_trace(trace, row=1, col=2)

# for trace in fig_family.data:
#     fig.add_trace(trace, row=2, col=1)

# for trace in fig_jaundice.data:
#     fig.add_trace(trace, row=2, col=2)

# fig.update_layout(title={'text': '', 'x': 0.5}, height=1000, width=2000)
# fig.show()

# Group 4 - Correlation
# fig = make_subplots(
#     rows=1, cols=2,
#     subplot_titles=(
#         'Correlation of Positive Responses with ASD Traits',
#         'Positive Response Rate for Individual Questions (A1-A10)',
#     ),
#     specs=[[{'type': 'bar'}, {'type': 'bar'}]]
# )
# for trace in fig_correlation.data:
#     fig.add_trace(trace, row=1, col=1)
```

```
# for trace in fig_bar_questions.data:
#     fig.add_trace(trace, row=1, col=2)

# fig.update_layout(title={'text': '', 'x': 0.5}, height=700, width=2000)
# fig.show()
```

## 1.2 Part 2: Data Cleaning

- Identify columns that are not needed for prediction
- Identify categorical columns or other columns that should be transformed

[1690]:
```
asd_toddlers_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1054 entries, 0 to 1053
Data columns (total 19 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Case_No                 1054 non-null   int64
 1   A1                      1054 non-null   int64
 2   A2                      1054 non-null   int64
 3   A3                      1054 non-null   int64
 4   A4                      1054 non-null   int64
 5   A5                      1054 non-null   int64
 6   A6                      1054 non-null   int64
 7   A7                      1054 non-null   int64
 8   A8                      1054 non-null   int64
 9   A9                      1054 non-null   int64
 10  A10                     1054 non-null   int64
 11  Age_Mons                1054 non-null   int64
 12  Qchat-10-Score          1054 non-null   int64
 13  Sex                     1054 non-null   object
 14  Ethnicity               1054 non-null   object
 15  Jaundice                1054 non-null   object
 16  Family_mem_with_ASD     1054 non-null   object
 17  Who completed the test  1054 non-null   object
 18  ASD_Traits              1054 non-null   object
dtypes: int64(13), object(6)
memory usage: 156.6+ KB
```

Based on the data available we can drop a couple columns and reformat others

[1691]:
```
# asd_toddlers_df.drop(columns=['Case_No', 'Who completed the test',␣
↪'Qchat-10-Score'], inplace=True)
# # convert yes/no to 1/0, m/f to 1/0
```

```
# asd_toddlers_df = asd_toddlers_df.replace({'yes': 1, 'no': 0, 'YES': 1, 'NO':␣
 ↪0, 'Yes': 1, 'No': 0, 'm': 1, 'f': 0})
# # split out ethnicity
# asd_toddlers_df = pd.get_dummies(asd_toddlers_df, columns=['Ethnicity'])
```

[1692]:
```
asd_toddlers_df.head()
asd_toddlers_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1054 entries, 0 to 1053
Data columns (total 19 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Case_No                 1054 non-null   int64
 1   A1                      1054 non-null   int64
 2   A2                      1054 non-null   int64
 3   A3                      1054 non-null   int64
 4   A4                      1054 non-null   int64
 5   A5                      1054 non-null   int64
 6   A6                      1054 non-null   int64
 7   A7                      1054 non-null   int64
 8   A8                      1054 non-null   int64
 9   A9                      1054 non-null   int64
 10  A10                     1054 non-null   int64
 11  Age_Mons                1054 non-null   int64
 12  Qchat-10-Score          1054 non-null   int64
 13  Sex                     1054 non-null   object
 14  Ethnicity               1054 non-null   object
 15  Jaundice                1054 non-null   object
 16  Family_mem_with_ASD     1054 non-null   object
 17  Who completed the test  1054 non-null   object
 18  ASD_Traits              1054 non-null   object
dtypes: int64(13), object(6)
memory usage: 156.6+ KB
```

[1693]:
```
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Check unique values for categorical features
categorical_cols = ['Sex', 'Ethnicity', 'Jaundice', 'Family_mem_with_ASD',␣
 ↪'ASD_Traits']
for col in categorical_cols:
    print(f"\nUnique values in {col}:")
    print(asd_toddlers_df[col].value_counts())

# Convert categorical variables to numerical
le_dict = {}
for col in categorical_cols:
```

```
    le = LabelEncoder()
    asd_toddlers_df[col] = le.fit_transform(asd_toddlers_df[col])
    le_dict[col] = le

# Convert target variable to numerical
le_target = LabelEncoder()
asd_toddlers_df['ASD_Traits'] = le_target.
 ↪fit_transform(asd_toddlers_df['ASD_Traits'])
print("\nTarget encoding:")
for i, label in enumerate(le_target.classes_):
    print(f"{label} -> {i}")

# Exclude Case_No, Qchat-10-Score, and who completed from features
asd_toddlers_df.drop(['Case_No', 'Qchat-10-Score', 'Who completed the test'],␣
 ↪axis=1, inplace=True)
```

```
Unique values in Sex:
Sex
m    735
f    319
Name: count, dtype: int64

Unique values in Ethnicity:
Ethnicity
White European    334
asian             299
middle eastern    188
south asian        60
black              53
Hispanic           40
Others             35
Latino             26
mixed               8
Pacifica            8
Native Indian       3
Name: count, dtype: int64

Unique values in Jaundice:
Jaundice
no     766
yes    288
Name: count, dtype: int64

Unique values in Family_mem_with_ASD:
Family_mem_with_ASD
no     884
yes    170
```

```
Name: count, dtype: int64

Unique values in ASD_Traits:
ASD_Traits
Yes    728
No     326
Name: count, dtype: int64

Target encoding:
0 -> 0
1 -> 1
```

[1694]:
```
asd_toddlers_df
```

[1694]:

| | A1 | A2 | A3 | A4 | A5 | A6 | A7 | A8 | A9 | A10 | Age_Mons | Sex | Ethnicity | \ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 28 | 0 | 8 | |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 36 | 1 | 5 | |
| 2 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 36 | 1 | 8 | |
| 3 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 24 | 1 | 0 | |
| 4 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 20 | 0 | 5 | |
| … | .. | .. | .. | .. | .. | .. | .. | .. | .. | … | … | … | … | |
| 1049 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 24 | 0 | 5 | |
| 1050 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 12 | 1 | 7 | |
| 1051 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 18 | 1 | 8 | |
| 1052 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 19 | 1 | 5 | |
| 1053 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 24 | 1 | 6 | |

| | Jaundice | Family_mem_with_ASD | ASD_Traits |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 |
| 2 | 1 | 0 | 1 |
| 3 | 0 | 0 | 1 |
| 4 | 0 | 1 | 1 |
| … | … | … | … |
| 1049 | 0 | 1 | 0 |
| 1050 | 1 | 0 | 1 |
| 1051 | 1 | 0 | 1 |
| 1052 | 0 | 1 | 0 |
| 1053 | 1 | 1 | 1 |

[1054 rows x 16 columns]

### 1.3 Part 3 - Creating a Deep Learning Model (PyTorch)

- Split the data and set up a DataSet and DataLoader
- Use PyTorch to create a simple deep neural network
- Train on split and validate

[1695]:
```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,␣
↪accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
```

[1696]:
```
# Set random seeds for reproducibility
torch.manual_seed(42)
np.random.seed(42)

X = asd_toddlers_df.drop(['ASD_Traits'], axis=1)
y = asd_toddlers_df['ASD_Traits'].values

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42, stratify=y)

# Standardize numerical features
numerical_cols = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'A7', 'A8', 'A9', 'A10',␣
↪'Age_Mons']
scaler = StandardScaler()
X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])
```

[1697]:
```
class AutismDataset(Dataset):
    def __init__(self, features, target):
        self.features = torch.tensor(features.values, dtype=torch.float32)
        self.target = torch.tensor(target, dtype=torch.long)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx], self.target[idx]

# Create data loaders
train_dataset = AutismDataset(X_train, y_train)
test_dataset = AutismDataset(X_test, y_test)

batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

```
test_loader = DataLoader(test_dataset, batch_size=batch_size)
```

[1698]:
```
class AutismClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim1=64, hidden_dim2=32, output_dim=2):
        super(AutismClassifier, self).__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim1)
        self.layer2 = nn.Linear(hidden_dim1, hidden_dim2)
        self.layer3 = nn.Linear(hidden_dim2, output_dim)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2)
        self.batch_norm1 = nn.BatchNorm1d(hidden_dim1)
        self.batch_norm2 = nn.BatchNorm1d(hidden_dim2)

    def forward(self, x):
        x = self.layer1(x)
        x = self.batch_norm1(x)
        x = self.relu(x)
        x = self.dropout(x)

        x = self.layer2(x)
        x = self.batch_norm2(x)
        x = self.relu(x)
        x = self.dropout(x)

        x = self.layer3(x)
        return x
```

[1699]:
```
input_dim = X_train.shape[1]
model = AutismClassifier(input_dim)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max',␣
 ↪factor=0.5, patience=3, verbose=True)

num_epochs = 50
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

def train_epoch(model, train_loader, criterion, optimizer):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, targets in train_loader:
```

```
        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, targets)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)

        # Calculate accuracy
        _, predicted = torch.max(outputs.data, 1)
        total += targets.size(0)
        correct += (predicted == targets).sum().item()

    epoch_loss = running_loss / total
    epoch_acc = correct / total
    return epoch_loss, epoch_acc

def validate(model, test_loader, criterion):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, targets in test_loader:
            outputs = model(inputs)
            loss = criterion(outputs, targets)

            running_loss += loss.item() * inputs.size(0)

            _, predicted = torch.max(outputs.data, 1)
            total += targets.size(0)
            correct += (predicted == targets).sum().item()

    epoch_loss = running_loss / total
    epoch_acc = correct / total
    return epoch_loss, epoch_acc

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(model, train_loader, criterion,␣
 ↪optimizer)
    val_loss, val_acc = validate(model, test_loader, criterion)

    train_losses.append(train_loss)
```

```
    val_losses.append(val_loss)
    train_accuracies.append(train_acc)
    val_accuracies.append(val_acc)

    # Update learning rate
    scheduler.step(val_acc)

    print(f'Epoch {epoch+1}/{num_epochs}: Train Loss: {train_loss:.4f}, Train␣
↪Acc: {train_acc:.4f}, '
          f'Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}')
```

```
Epoch 1/50: Train Loss: 0.6041, Train Acc: 0.6548, Val Loss: 0.4324, Val Acc:
0.8673
Epoch 2/50: Train Loss: 0.3642, Train Acc: 0.8778, Val Loss: 0.2729, Val Acc:
0.9384
Epoch 3/50: Train Loss: 0.2704, Train Acc: 0.9241, Val Loss: 0.2168, Val Acc:
0.9668
Epoch 4/50: Train Loss: 0.2301, Train Acc: 0.9359, Val Loss: 0.1830, Val Acc:
0.9668
```

```
/Users/owenmccormack/Desktop/Grad_School_Projects/Spring_2025/AI_In_Healthcare/A
ssignments/SELF_LEARNING_TUTORIAL/.venv/lib/python3.11/site-
packages/torch/optim/lr_scheduler.py:62: UserWarning:

The verbose parameter is deprecated. Please use get_last_lr() to access the
learning rate.
```

```
Epoch 5/50: Train Loss: 0.1864, Train Acc: 0.9490, Val Loss: 0.1428, Val Acc:
0.9763
Epoch 6/50: Train Loss: 0.1645, Train Acc: 0.9549, Val Loss: 0.1173, Val Acc:
0.9810
Epoch 7/50: Train Loss: 0.1416, Train Acc: 0.9561, Val Loss: 0.1067, Val Acc:
0.9953
Epoch 8/50: Train Loss: 0.1315, Train Acc: 0.9549, Val Loss: 0.0920, Val Acc:
0.9858
Epoch 9/50: Train Loss: 0.1177, Train Acc: 0.9597, Val Loss: 0.0831, Val Acc:
0.9905
Epoch 10/50: Train Loss: 0.1059, Train Acc: 0.9632, Val Loss: 0.0776, Val Acc:
0.9953
Epoch 11/50: Train Loss: 0.1104, Train Acc: 0.9573, Val Loss: 0.0704, Val Acc:
0.9953
Epoch 12/50: Train Loss: 0.1157, Train Acc: 0.9526, Val Loss: 0.0701, Val Acc:
0.9953
Epoch 13/50: Train Loss: 0.0883, Train Acc: 0.9715, Val Loss: 0.0698, Val Acc:
0.9858
Epoch 14/50: Train Loss: 0.1165, Train Acc: 0.9573, Val Loss: 0.0648, Val Acc:
0.9953
Epoch 15/50: Train Loss: 0.0880, Train Acc: 0.9715, Val Loss: 0.0613, Val Acc:
```

```
0.9953
Epoch 16/50: Train Loss: 0.0764, Train Acc: 0.9786, Val Loss: 0.0637, Val Acc:
0.9953
Epoch 17/50: Train Loss: 0.0769, Train Acc: 0.9727, Val Loss: 0.0628, Val Acc:
0.9953
Epoch 18/50: Train Loss: 0.1184, Train Acc: 0.9585, Val Loss: 0.0665, Val Acc:
0.9905
Epoch 19/50: Train Loss: 0.0791, Train Acc: 0.9715, Val Loss: 0.0607, Val Acc:
0.9953
Epoch 20/50: Train Loss: 0.1012, Train Acc: 0.9620, Val Loss: 0.0573, Val Acc:
0.9953
Epoch 21/50: Train Loss: 0.0810, Train Acc: 0.9751, Val Loss: 0.0591, Val Acc:
0.9953
Epoch 22/50: Train Loss: 0.0799, Train Acc: 0.9680, Val Loss: 0.0611, Val Acc:
0.9953
Epoch 23/50: Train Loss: 0.0755, Train Acc: 0.9739, Val Loss: 0.0600, Val Acc:
0.9953
Epoch 24/50: Train Loss: 0.0838, Train Acc: 0.9727, Val Loss: 0.0556, Val Acc:
0.9953
Epoch 25/50: Train Loss: 0.0791, Train Acc: 0.9751, Val Loss: 0.0614, Val Acc:
0.9953
Epoch 26/50: Train Loss: 0.0855, Train Acc: 0.9703, Val Loss: 0.0625, Val Acc:
0.9905
Epoch 27/50: Train Loss: 0.0738, Train Acc: 0.9810, Val Loss: 0.0563, Val Acc:
0.9953
Epoch 28/50: Train Loss: 0.0903, Train Acc: 0.9692, Val Loss: 0.0559, Val Acc:
0.9953
Epoch 29/50: Train Loss: 0.0876, Train Acc: 0.9656, Val Loss: 0.0562, Val Acc:
0.9953
Epoch 30/50: Train Loss: 0.0780, Train Acc: 0.9668, Val Loss: 0.0552, Val Acc:
0.9953
Epoch 31/50: Train Loss: 0.0768, Train Acc: 0.9798, Val Loss: 0.0569, Val Acc:
0.9953
Epoch 32/50: Train Loss: 0.0840, Train Acc: 0.9656, Val Loss: 0.0570, Val Acc:
0.9953
Epoch 33/50: Train Loss: 0.0993, Train Acc: 0.9680, Val Loss: 0.0546, Val Acc:
0.9953
Epoch 34/50: Train Loss: 0.0830, Train Acc: 0.9680, Val Loss: 0.0574, Val Acc:
0.9953
Epoch 35/50: Train Loss: 0.0837, Train Acc: 0.9692, Val Loss: 0.0568, Val Acc:
0.9953
Epoch 36/50: Train Loss: 0.1190, Train Acc: 0.9442, Val Loss: 0.0576, Val Acc:
0.9953
Epoch 37/50: Train Loss: 0.0765, Train Acc: 0.9786, Val Loss: 0.0560, Val Acc:
0.9953
Epoch 38/50: Train Loss: 0.0786, Train Acc: 0.9727, Val Loss: 0.0552, Val Acc:
0.9953
Epoch 39/50: Train Loss: 0.0961, Train Acc: 0.9609, Val Loss: 0.0551, Val Acc:
```

```
0.9953
Epoch 40/50: Train Loss: 0.0961, Train Acc: 0.9620, Val Loss: 0.0562, Val Acc:
0.9953
Epoch 41/50: Train Loss: 0.1202, Train Acc: 0.9502, Val Loss: 0.0560, Val Acc:
0.9953
Epoch 42/50: Train Loss: 0.1053, Train Acc: 0.9609, Val Loss: 0.0569, Val Acc:
0.9953
Epoch 43/50: Train Loss: 0.0844, Train Acc: 0.9715, Val Loss: 0.0550, Val Acc:
0.9953
Epoch 44/50: Train Loss: 0.0755, Train Acc: 0.9775, Val Loss: 0.0546, Val Acc:
0.9953
Epoch 45/50: Train Loss: 0.0940, Train Acc: 0.9609, Val Loss: 0.0566, Val Acc:
0.9953
Epoch 46/50: Train Loss: 0.0755, Train Acc: 0.9763, Val Loss: 0.0582, Val Acc:
0.9953
Epoch 47/50: Train Loss: 0.0788, Train Acc: 0.9763, Val Loss: 0.0558, Val Acc:
0.9953
Epoch 48/50: Train Loss: 0.1071, Train Acc: 0.9597, Val Loss: 0.0556, Val Acc:
0.9953
Epoch 49/50: Train Loss: 0.0851, Train Acc: 0.9644, Val Loss: 0.0545, Val Acc:
0.9953
Epoch 50/50: Train Loss: 0.0745, Train Acc: 0.9715, Val Loss: 0.0546, Val Acc:
0.9953
```

[ ]:
```python
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs + 1), train_losses, label='Train Loss')
plt.plot(range(1, num_epochs + 1), val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()

# Plot training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs + 1), train_accuracies, label='Train Accuracy')
plt.plot(range(1, num_epochs + 1), val_accuracies, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.tight_layout()
plt.savefig('training_history.png')
plt.close()

# Generate predictions on the test set
model.eval()
```

```
all_preds = []
all_targets = []

with torch.no_grad():
    for inputs, targets in test_loader:
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)

        all_preds.extend(predicted.cpu().numpy())
        all_targets.extend(targets.cpu().numpy())

# Convert predictions and targets back to original labels
all_preds_labels = le_target.inverse_transform(all_preds)
all_targets_labels = le_target.inverse_transform(all_targets)

# Print classification report
print("\nClassification Report:")
target_names = [str(class_name) for class_name in le_target.classes_]
print(classification_report(all_targets, all_preds, target_names=target_names))

# Create confusion matrix
cm = confusion_matrix(all_targets, all_preds)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=le_target.
 ↪classes_, yticklabels=le_target.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.savefig('confusion_matrix.png')
plt.close()
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        65
           1       1.00      0.99      1.00       146

    accuracy                           1.00       211
   macro avg       0.99      1.00      0.99       211
weighted avg       1.00      1.00      1.00       211
```

[1701]:
```
# 6. Feature Importance Analysis
# We can analyze feature importance by looking at the weights of the first layer
importance = model.layer1.weight.data.abs().mean(dim=0).cpu().numpy()
feature_names = X.columns.tolist()
```

```
# Create a DataFrame to hold feature importance
importance_asd_toddlers_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importance
})
importance_asd_toddlers_df = importance_asd_toddlers_df.
 ↪sort_values('Importance', ascending=False)

# Plot feature importance
plt.figure(figsize=(10, 8))
sns.barplot(x='Importance', y='Feature', data=importance_asd_toddlers_df)
plt.title('Feature Importance')
plt.tight_layout()
plt.savefig('feature_importance.png')
plt.close()

print("\nFeature Importance:")
for i, row in importance_asd_toddlers_df.iterrows():
    print(f"{row['Feature']}: {row['Importance']:.4f}")
```

```
Feature Importance:
A9: 0.1503
A2: 0.1477
Jaundice: 0.1372
A5: 0.1371
A8: 0.1350
A6: 0.1341
A10: 0.1339
A1: 0.1319
A4: 0.1307
A3: 0.1290
A7: 0.1278
Family_mem_with_ASD: 0.1276
Sex: 0.1242
Age_Mons: 0.1188
Ethnicity: 0.0990
```

[ ]:
```
torch.save({
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scaler': scaler,
    'label_encoders': le_dict,
    'target_encoder': le_target,
    'feature_names': feature_names
}, 'autism_classifier_model.pth')
```